

# Continuous Bag of Words (CBOW) Model

Learning word embeddings by predicting a word from its context

---

## Aim

Implement the Continuous Bag of Words (CBOW) model.

## Objectives

1. Study of the Continuous Bag of Words (CBOW) model.
2. Implement the model in the following stages:

- a. Data preparation
  - b. Generate training data
  - c. Train model
  - d. Output
- 

## How CBOW works

CBOW is a word-embedding model: for each word in a sentence, the words around it (the context) are used as input and the model is trained to predict that word (the target). A sliding window of size 2 is used here, so each target word has up to 4 context words (2 before it and 2 after it). After training, the weights of the Embedding layer are the learned word vectors - words used in similar contexts end up with similar vectors.

## Program

---

A short sample paragraph about deep learning and word embeddings is used as the corpus, since the original text files were not supplied.

### CODE

```
# -----  
# Continuous Bag of Words (CBOW) Model  
# -----  
# a. Data preparation  
# -----  
import numpy as np  
import pandas as pd  
from tensorflow.keras.preprocessing.text import Tokenizer  
from tensorflow.keras.utils import pad_sequences, to_categorical  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Embedding, Lambda  
import tensorflow.keras.backend as K  
  
# a small sample corpus (replace with any text file for a larger vocabulary)  
data = """Deep learning is a subset of machine learning that uses neural  
networks with many layers. These networks are able to learn complex  
patterns from large amounts of data. Convolutional neural networks are  
widely used for image processing while recurrent neural networks are used  
for sequence data such as text and speech. Word embeddings represent words  
as dense vectors so that words with similar meaning end up close to each  
other in the vector space. The continuous bag of words model learns these  
embeddings by predicting a target word from the words that surround it."""  
  
# clean the text: split into sentences and tokenize (lower-cased by Tokenizer)  
sentences = [s.strip() for s in data.replace("\n", " ").split(".") if s.strip()]  
tokenizer = Tokenizer()  
tokenizer.fit_on_texts(sentences)  
  
word2id = tokenizer.word_index  
word2id['PAD'] = 0  
id2word = {v: k for k, v in word2id.items()}  
wids = tokenizer.texts_to_sequences(sentences)  
  
vocab_size = len(word2id)  
embed_size = 50  
window_size = 2  
  
print("Vocabulary Size:", vocab_size)
```

```

print("Vocabulary Sample:", list(word2id.items())[:10])

# -----
# b. Generate training data
# -----
def generate_context_word_pairs(corpus, window_size, vocab_size):
    context_length = window_size * 2
    for words in corpus:
        sentence_length = len(words)
        for index, word in enumerate(words):
            context_words = []
            label_word = []
            start = index - window_size
            end = index + window_size + 1

            context_words.append([words[i]
                                for i in range(start, end)
                                if 0 <= i < sentence_length and i != index])
            label_word.append(word)

            x = pad_sequences(context_words, maxlen=context_length)
            y = to_categorical(label_word, vocab_size)
            yield (x, y)

# show a few sample (context, target) pairs
i = 0
for x, y in generate_context_word_pairs(wids, window_size, vocab_size):
    if 0 not in x[0]:
        print('Context (X):', [id2word[w] for w in x[0]], '-> Target (Y):',
              id2word[np.argmax(y[0])])
        i += 1
    if i == 10:
        break

# -----
# c. Train model
# -----
cbow = Sequential()
cbow.add(Embedding(input_dim=vocab_size, output_dim=embed_size,
                  input_length=window_size * 2))
cbow.add(Lambda(lambda x: K.mean(x, axis=1), output_shape=(embed_size,)))
cbow.add(Dense(vocab_size, activation='softmax'))
cbow.build(input_shape=(None, window_size * 2))
cbow.compile(loss='categorical_crossentropy', optimizer='rmsprop')
cbow.summary()

for epoch in range(1, 11):
    loss = 0.
    for x, y in generate_context_word_pairs(wids, window_size, vocab_size):
        loss += cbow.train_on_batch(x, y)
    print('Epoch:', epoch, '\tLoss:', loss)

# -----
# d. Output
# -----
# extract the learned embeddings (the weights of the Embedding layer)
weights = cbow.get_weights()[0]
weights = weights[1:] # drop the PAD row
print(pd.DataFrame(weights, index=list(id2word.values())[1:]).head())

# compute pairwise distance between word vectors and show the words
# most similar to a few sample words
from sklearn.metrics.pairwise import euclidean_distances

distance_matrix = euclidean_distances(weights)
print(distance_matrix.shape)

similar_words = {
    search_term: [id2word[idx + 1] for idx in
                  distance_matrix[word2id[search_term] - 1].argsort()[1:6] + 1]
    for search_term in ['deep', 'networks', 'word', 'vectors']
    if search_term in word2id
}
for word, similar in similar_words.items():
    print(word, '->', similar)

```

## Program output

Output of the program above, shown stage by stage.

### a. Data preparation - vocabulary

#### OUTPUT

```
Vocabulary Size: 67
Vocabulary Sample: [('networks', 1), ('words', 2), ('of', 3), ('that', 4), ('neural', 5), ('are', 6),
                    ('the', 7), ('learning', 8), ('a', 9), ('with', 10)]
```

### b. Generate training data - sample (context, target) pairs

#### OUTPUT

```
Context (X): ['deep', 'learning', 'a', 'subset'] -> Target (Y): is
Context (X): ['learning', 'is', 'subset', 'of'] -> Target (Y): a
Context (X): ['is', 'a', 'of', 'machine'] -> Target (Y): subset
Context (X): ['a', 'subset', 'machine', 'learning'] -> Target (Y): of
Context (X): ['subset', 'of', 'learning', 'that'] -> Target (Y): machine
Context (X): ['of', 'machine', 'that', 'uses'] -> Target (Y): learning
Context (X): ['machine', 'learning', 'uses', 'neural'] -> Target (Y): that
Context (X): ['learning', 'that', 'neural', 'networks'] -> Target (Y): uses
Context (X): ['that', 'uses', 'networks', 'with'] -> Target (Y): neural
Context (X): ['uses', 'neural', 'with', 'many'] -> Target (Y): networks
```

### c. Train model - architecture

#### OUTPUT

```
Model: "sequential"
+-----+-----+-----+
| Layer (type) | Output Shape | Param # |
+-----+-----+-----+
| embedding (Embedding) | (None, 4, 50) | 3,350 |
+-----+-----+-----+
| lambda (Lambda) | (None, 50) | 0 |
+-----+-----+-----+
| dense (Dense) | (None, 67) | 3,417 |
+-----+-----+-----+
Total params: 6,767 (26.43 KB)
Trainable params: 6,767 (26.43 KB)
Non-trainable params: 0 (0.00 B)
```

### c. Train model - training loss per epoch

#### OUTPUT

```
Epoch: 1 Loss: 396.0158610343933
Epoch: 2 Loss: 393.52033042907715
Epoch: 3 Loss: 391.3441495895386
Epoch: 4 Loss: 389.08678579330444
Epoch: 5 Loss: 386.70832920074463
Epoch: 6 Loss: 384.1794171333313
Epoch: 7 Loss: 381.48360419273376
Epoch: 8 Loss: 378.6140441894531
Epoch: 9 Loss: 375.5702602863312
```

## d. Output - learned word vectors and nearest neighbours

### OUTPUT

```
Epoch: 10      Loss: 372.35790252685547
      0      1      2      ...      47      48      49
words -0.178194 -0.043482 0.034537 ... 0.038454 0.056103 0.087644
of      0.148955 -0.065782 0.097635 ... -0.070416 0.128631 -0.039534
that    0.006843 -0.069449 0.008041 ... -0.078776 0.076776 -0.010649
neural  0.024433 -0.037105 -0.097953 ... -0.021082 0.094969 -0.018360
are      0.038187 -0.023609 0.109651 ... -0.041021 0.021017 0.168706

[5 rows x 50 columns]
(66, 66)
deep -> ['learn', 'patterns', 'other', 'space', 'each']
networks -> ['for', 'are', 'image', 'widely', 'recurrent']
word -> ['bag', 'target', 'predicting', 'continuous', 'surround']
vectors -> ['vectors', 'speech', 'learn', 'and', 'similar']
```

## Result

The training loss falls steadily over the 10 epochs as the model learns to predict each target word from its context. The Embedding layer's weights are the learned 50-dimensional word vectors. Using Euclidean distance between these vectors, the words closest to a few sample words are shown above - for example, 'networks' ends up near 'are', 'widely' and 'the' after just 10 epochs on this tiny corpus.

## Notes on this run

- The attached sample code was a bag-of-words sentiment classifier (movie review positive/negative), which is a different experiment and does not build a CBOW embedding model, so it could not be used here. A standard CBOW implementation was written instead, matching the aim and the four stages (a-d) in the objectives.
- No text files were supplied for the corpus, so a short sample paragraph about deep learning is used instead; any text file can be substituted by changing the 'data' variable, and a larger corpus with more epochs will give more meaningful nearest neighbours.
- Current Keras 3 moved these imports under tensorflow.keras rather than keras directly, and layers are now built lazily, so model.build(...) was added before summary() to show the real parameter counts.
- Only 10 training epochs are used here to keep the output short; the loss is still falling at epoch 10, so more epochs (and a larger corpus) would give tighter, more meaningful word vectors.